DART APPRENTICE

The Real-World Engineering Challenge

Serious console-programming challenges based only on the chapters covered so far

---

Based on: *Dart Apprentice: Fundamentals* by Jonathan Sande

Coverage: fundamentals, expressions and variables, types and operations, strings, control flow, loops, functions, and classes.

Offline challenge workbook

The Dart Apprentice: The Real-World Engineering Challenge

SLIIIIIME! You said **SERIOUS CHALLENGES**, so we're not doing another calculator that adds two numbers and calls itself a project. ■

You've covered enough Dart to start solving meaningful problems with console applications. The key now is to stop merely recognizing syntax and start making engineering decisions: how to represent data, validate input, model real-world situations, handle edge cases, and structure logic.

I'm going to give you a proper challenge set based on the eight areas you've studied in *Dart Apprentice: Fundamentals* by Jonathan Sande.

The rule: We will use only what you've learned so far. No Flutter, no databases, no APIs, no packages, no file handling, no lists or maps unless you've already studied them, and no advanced Dart concepts from later chapters.

01. Small Business Sales & Profit Intelligence System

Level 1 · Foundation

Build a console application for a Malawian small business that needs to know whether it is actually making money.

The real problem: Many small businesses confuse cash received with profit earned.

Your system must:

- Store a business name, product name, selling price, purchase cost, and quantity sold.
- Calculate total sales revenue, total cost of goods sold, gross profit, and gross profit margin.
- Calculate discounts and determine how they affect profitability.
- Use conditional logic to classify the result as a loss, break-even, or profit.
- Reject invalid quantities and negative prices.
- Use functions for calculations and a class to represent a product or sale.
- Print a clean business report using string interpolation and formatted output.

Serious edge cases:

- The business sells nothing.
- The selling price equals the purchase cost.
- A discount makes the sale unprofitable.
- The quantity is zero or negative.
- The cost is higher than the selling price.

Acceptance test
A trader buys 12 items at MWK 4,500 each and sells them at MWK 6,000 each, with a 10% discount on the total selling price.

Your program must calculate:
- Revenue after discount: MWK 64,800
- Cost of goods sold: MWK 54,000
- Gross profit: MWK 10,800
- Gross profit margin: 16.67% approximately

If your output is wrong, your system has failed the test.

Concepts tested: Variables, constants, types, arithmetic, strings, control flow, functions, loops, and classes.

02. Farm Operations & Investment Decision Engine

Level 2 · Applied logic

Build a system that helps a farm manager decide whether a proposed crop investment makes financial sense.

The real problem: A farm has limited money and must estimate costs, revenue, and expected returns before committing funds.

Your system must:

- Represent a crop using a class containing its name, cultivated area, expected yield per acre, selling price, and production cost.
- Calculate expected harvest, gross revenue, total costs, net profit, and return on investment (ROI).
- Calculate the minimum selling price required to break even.
- Compare two crop investment proposals using separate objects and functions.
- Use loops to evaluate several hypothetical yield or price scenarios.
- Issue warnings when expected profit is negative or the budget is exceeded.

Serious edge cases:

- A crop fails completely and produces zero yield.
- Market prices fall by 20%.
- Production costs rise by 15%.
- Available capital is less than the proposed investment.
- ROI cannot be calculated because the investment is zero.

Acceptance test
A farmer invests MWK 1,200,000, expects to sell the harvest for MWK 1,800,000, and estimates total production costs at MWK 1,200,000.

Your system should report:
- Revenue: MWK 1,800,000
- Net profit: MWK 600,000
- ROI on production cost: 50%

Then simulate a 20% drop in selling price. Your program must recalculate profitability rather than reuse the original result.

Concepts tested: Classes, constructors if covered, numeric types, constants, functions, loops, conditions, and string interpolation.

03. Mobile Money Transaction Risk Simulator

Level 3 · Financial logic

Build a simplified transaction engine for a hypothetical mobile money service.

The real problem: A payment system must validate transactions, calculate fees, enforce limits, and avoid approving invalid transfers.

Your system must:

- Model an account with an account holder's name, account number, and balance.
- Calculate a transfer fee using a specified fee rate and minimum fee.
- Validate the transfer amount before attempting a transaction.
- Check the account balance, transfer limits, and transaction status.
- Deduct the transfer amount and fee only when the transaction is approved.
- Use a transaction class to represent a transfer and its outcome.
- Generate a receipt showing the amount, fee, total debit, recipient, and remaining balance.
- Process a sequence of hypothetical transactions using a loop.

Serious edge cases:

- Zero or negative transfer amount.
- Insufficient funds to cover the transfer and fee.
- Transfer exceeds the transaction limit.
- Balance exactly equals the total debit.
- A rejected transfer must not alter the balance.

Acceptance test
Starting balance: MWK 50,000. Transfer amount: MWK 20,000. Fee: 2% of the transfer amount.

Expected result:
- Transfer amount: MWK 20,000
- Fee: MWK 400
- Total debit: MWK 20,400
- Remaining balance: MWK 29,600

Now attempt a transfer of MWK 40,000. The system must reject it and preserve the balance at MWK 29,600.

Concepts tested: Classes, state changes, arithmetic, validation, functions, conditions, loops, and formatted receipts.

This is a learning simulator, not a production payment system. Real transaction systems need persistence, authentication, concurrency control, and other protections that are outside your current syllabus.

---

04. Student Academic Performance & Progress Analyzer

Level 3 · Data processing

Build a system for a lecturer who needs to evaluate a student's academic performance across several assessments.

The real problem: Marks need to be validated, weighted correctly, and translated into meaningful academic feedback.

Your system must:

- Create a student class containing identifying information and relevant academic details.
- Accept marks for several assessments through a defined set of variables.

- Validate that marks fall within the permitted range.
- Calculate weighted coursework and examination scores.
- Determine the final grade using configurable grade boundaries.
- Use a loop to simulate marks from 0 to 100 and evaluate the grade classification.
- Generate a report with the final score, grade, pass/fail status, and feedback.

Serious edge cases:

- Marks below zero or above 100.
- A student scores exactly on a grade boundary.
- A coursework score is missing or invalid.
- A student passes coursework but fails the examination.
- Decimal marks create rounding differences.

Acceptance test
Suppose coursework contributes 40% and the examination contributes 60%.

A student earns 75% in coursework and 65% in the examination.

Expected final score:
75(0.40) + 65(0.60) = 69%

Your grade must follow the grading rules you define and document, not a classification you invent halfway through the program.

Concepts tested: Types, numeric operations, classes, functions, loops, conditionals, and strings.

---

05. Inventory Reorder & Stockout Prevention System

Level 4 · Business rules

Build an inventory controller for a retail shop that needs to prevent stockouts and avoid unnecessary purchases.

The real problem: The shop must know when to reorder, how much to buy, and whether the purchase fits its budget.

Your system must:

- Model an inventory item with its name, stock quantity, reorder threshold, unit cost, and target stock level.
- Calculate stock remaining after sales.
- Determine whether the item needs reordering.
- Calculate the reorder quantity and estimated procurement cost.
- Check whether the procurement cost fits the available budget.
- Use a loop to simulate daily sales and generate stock alerts.
- Print a stock report with status classifications.

Serious edge cases:

- Stock reaches exactly zero.
- A sale requests more stock than is available.
- The reorder threshold equals the current stock.

- The available budget cannot cover the recommended order.
- A reorder quantity is zero or negative because the configuration is invalid.

Acceptance test
Current stock: 18 units. Reorder threshold: 10 units. Target stock: 50 units. Unit cost: MWK 2,500.

After selling 9 units:
- Remaining stock: 9 units
- Reorder status: Required
- Recommended order: 41 units
- Procurement cost: MWK 102,500

With a procurement budget of MWK 100,000, your system must report a budget shortfall of MWK 2,500.

Concepts tested: Classes, variables, arithmetic, loops, functions, conditional logic, and validation.

Final boss: Build a mini business management suite

Level 5 · Capstone

Now combine the ideas from the previous challenges into one console application for a fictional Malawian business.

The business has:

- A starting cash balance.
- Sales revenue.
- Operating expenses.
- Inventory purchases.
- Customer payments and transfers.
- A target profit for the period.

Your job is to build a **Business Operations Simulator** that can process a sequence of predefined operations and produce a final business report.

Required features

- **Object-oriented modelling:** Create suitable classes for the business, products, and transactions.
- **Business calculations:** Calculate revenue, expenses, cash balance, inventory costs, and net profit using clearly defined accounting assumptions.
- **Transaction processing:** Approve or reject transactions according to the business rules you define.
- **Control flow:** Use conditions to classify business performance and identify financial risks.
- **Loops:** Process multiple hypothetical operations without duplicating the same code.
- **Functions:** Separate validation, calculations, transaction processing, and reporting into reusable functions.
- **Input validation:** Reject invalid values without corrupting the business state.
- **Reporting:** Print an understandable summary showing what happened and why.

Capstone acceptance tests

- A profitable sales period.
- A loss-making period.
- A transaction rejected for insufficient funds.
- An invalid operation that leaves the business state unchanged.
- A zero-sales period.
- An expense that reduces cash below a defined warning threshold.
- A final report that reconciles the opening balance, cash inflows, cash outflows, and closing balance.

Important distinction: Cash balance and net profit are not the same thing. For example, buying inventory can reduce cash immediately without making the entire purchase an expense for profit calculations at that moment. Define your accounting assumptions clearly so that the same amount is not counted twice.

Your engineering rules

Slime, this is where the learning gets serious. Follow these rules for every challenge.

Rule 1 — No tutorial copying
Write the solution yourself. You can consult your book for syntax, but design the program before asking for code.

Rule 2 — Separate responsibilities
Avoid placing every calculation inside main(). Use functions and classes for distinct responsibilities.

Rule 3 — Test boundaries, not just happy paths
Test zero, negative values, exact limits, invalid configurations, and operations that should fail.

Rule 4 — Defend your design
You should be able to explain why you selected each data type, created each class, and separated each function.

Rule 5 — Stay inside the syllabus
Use only the chapters you've covered. If you haven't learned collections, exception handling, enums, or file I/O yet, don't use them to bypass the challenge. Where repeated records are required, use a fixed set of objects and loop over predefined scenarios only if the syntax has been covered.

How to submit your work

For each challenge, provide:

- The problem analysis and assumptions.
- The class design and variables you intend to use.
- The complete Dart source code.
- Sample output from a normal case and at least three edge cases.
- A short explanation of one bug you encountered and how you fixed it.

Your challenge tracker

Use the checklist below to track progress offline. Tick each challenge when you consider it complete.

Done	Challenge	Focus
■	01 · Sales & Profit Intelligence	Revenue, discounts, margin, and validation
■	02 · Farm Investment Engine	Yield, ROI, break-even, and scenarios
■	03 · Mobile Money Simulator	Transfers, fees, limits, and state changes
■	04 · Academic Performance Analyzer	Weighted marks, boundaries, and reports
■	05 · Inventory Reorder System	Stock movements, procurement, and budget
■	06 · Business Operations Capstone	Integrate the systems into one application

My recommendation

Start with Challenge 01, but don't underestimate it. A clean sales system that handles discounts, profit margins, invalid data, and zero-value cases correctly is a much better demonstration of understanding than a much larger program full of tangled logic.

When you finish it, bring me your Dart code. I'll review it like a software engineering mentor: correctness, class design, function responsibilities, edge cases, readability, and whether you've genuinely applied the concepts rather than merely made the program run.

We're training you to think like a developer now, slime. Not just to write Dart syntax. ■